

Inheritance in Python - Interview Notes

Inheritance Notes

1. Definition:

Inheritance allows a class (child) to acquire properties and methods of another class (parent). It promotes reusability and hierarchical relationships.

2. Why Use It?

- Avoid code repetition
- Extend existing functionality
- Build class hierarchies

3. Syntax Example:

```
class Parent:
```

```
    def display(self):  
        print("Parent class")
```

```
class Child(Parent):
```

```
    def show(self):  
        print("Child class")
```

```
obj = Child()
```

```
obj.display()
```

```
obj.show()
```

4. Types of Inheritance:

- Single
- Multiple
- Multilevel
- Hierarchical
- Hybrid

Inheritance in Python - Interview Notes

Example: Single Inheritance

class A:

```
def feature1(self):  
    print("Feature 1")
```

class B(A):

```
def feature2(self):  
    print("Feature 2")
```

obj = B()

obj.feature1()

obj.feature2()

5. Method Overriding:

class Parent:

```
def greet(self):  
    print("Hello from Parent")
```

class Child(Parent):

```
def greet(self):  
    print("Hello from Child")
```

obj = Child()

obj.greet()

6. Using super():

class Parent:

```
def show(self):  
    print("Parent method")
```

Inheritance in Python - Interview Notes

```
class Child(Parent):  
    def show(self):  
        super().show()  
        print("Child method")
```

7. Real-world Example:

```
class Employee:  
    def __init__(self, name, salary):  
        self.name = name  
        self.salary = salary  
  
class Manager(Employee):  
    def __init__(self, name, salary, dept):  
        super().__init__(name, salary)  
        self.dept = dept
```

```
m1 = Manager("Sai Kiran", 75000, "Data Analytics")
```

8. Interview Key Points:

- Inheritance = Code Reusability
- super() = Calls parent constructor/method
- MRO = Defines method search order
- Types = Single, Multiple, Multilevel, Hierarchical, Hybrid